Performance budgets

Real, measured numbers behind acceptance criteria that would otherwise be unfalsifiable prose ("large worktrees stay responsive," "the app feels fast"). Each entry states what was measured, on what, and a concrete number a future change can be measured against — and can fail. Add a section per endpoint/ subsystem as they get their own criterion; do not fold this into `docs/RELEASE_GATES.md` (which is about which checks gate a release, a different question) or `docs/GIT_CLIENT_ROADMAP.md` (a milestone roadmap, not a measurement log).

Caveat that applies to every number in this file, stated once here rather than repeated per section: these are single runs on one development host (4 cores, this box, debug/unoptimized build unless a section says otherwise), not a statistically controlled benchmark suite — no warm-up runs, no repeated trials, no variance reported. Treat every number here as "real and reproducible," not "precise to the millisecond." A regression test derived from a budget uses a generous multiple of the measured number specifically because of this — see each section's own regression-test note.

`GET /api/status/v2` (#68e, M2.15)

What was measured. The real handler seam (`worktree_status_v2_for_repo` in `crates/git-vista-server/src/handlers/ read.rs`) end to end: the `git status --porcelain=v2 --branch -z` spawn, the `-z` porcelain read through the capped primitive, `parse_porcelain_v2_z` (#68b), and the full generation derivation (`read_generation_inputs`'s HEAD + every ref + index walk, plus the sha256 digest of the read bytes, #68c). Not just the git spawn alone — the generation derivation walks every ref in the repository too, and isolating only the parse would have missed that as a real cost center.

Reproduce with:

```
cargo test -p git-vista-server --bin git-vista-server -- --ignored --nocapture \
  large_worktree_responsiveness_ladder
cargo test -p git-vista-server --bin git-vista-server -- --ignored --nocapture \
  large_worktree_cap_boundary_in_file_count
```

(Both `#[ignore]`d — real file generation and repeated git spawns, no place in every `cargo test`/CI run. See their doc comments in `read.rs`.)

The ladder — worktree size vs. wall-clock time

Untracked files (`generate_untracked_files`), debug build, one run each, this host:

files changed	elapsed	cap hit?
100	4.413 ms	no
1,000	8.963 ms	no
5,000	17.800 ms	no
20,000	64.864 ms	no

Roughly linear in file count across two orders of magnitude — no evidence of a superlinear cost center (e.g. an accidentally quadratic ref walk or parse step) at these sizes. Untracked entries are the cheapest real record shape (no mode/hash fields to compute per entry, unlike a staged or unstaged change), so this ladder is closer to a best case than a worst case for a given file count — a worktree with the same file count but every file modified (carrying real mode/hash fields per porcelain record) would cost somewhat more per file, not measured separately here.

Where the 8 MiB read cap actually bites, in file count

Task 13 (#68c) chose an 8 MiB cap (`STATUS_V2_STDOUT_CAP`) and made a cap hit a `413`, not a best-effort parse. Nobody had measured what worktree size that actually corresponds to. Measured: **450,000 untracked files with uniform 15-character names** (`bench-NNNNNN.txt`, `?` + name + NUL = 20 bytes/record ⇒ ~8.6 MiB) reliably exceeds the cap — confirmed by `large_worktree_cap_boundary_in_file_count` asserting the call is refused, not corrupted-parsed. Total wall time for that run (dominated by creating 450,000 real files, not by `git status` itself) was **72.16 s**.

This is a lower bound for this specific naming scheme, not a universal constant. A real worktree's actual paths are typically longer (directory nesting, real filenames) than a 15-character uniform name, and every non-untracked record (staged, unstaged, renamed, conflicted) carries mode and hash fields the untracked shape doesn't — both push the real-world file count that trips the cap down from 450,000, not up. Treat 450,000 as "the cap does not bite for any worktree size anyone has actually filed an issue about," not as "this is exactly where real users will hit it."

Stated budget

A `GET /api/status/v2` request against a worktree with up to 20,000 changed files must complete in well under 1 second on hardware comparable to this host. The measured 64.9 ms at 20,000 files leaves roughly 15x headroom before that budget — deliberately generous, not a tight fit to today's number, so the regression test below doesn't flake on a loaded CI runner while still catching a real regression.

Regression test: `worktree_status_v2_budget_holds_at_1k_files` in `read.rs` (not `#[ignore]`d — runs in every `cargo test`) asserts 1,000 changed files complete inside **2 seconds** — roughly 220x the measured 8.96 ms at that size, chosen loose enough to tolerate a slow/loaded runner while still catching an actual regression (e.g. the generation derivation's ref walk becoming accidentally quadratic, which would show up as a multi-second stall at a mere 1,000 files, nowhere close to this budget).

`hunk_nav` — diff view hunk-navigation walk (partial progress on #211, M2.16f)

Status: partial delivery, #211 stays open. This section covers 1 of

211's 4 scope items (a regression test) against a substitute target, not

the one the issue names. It does **not** measure "the virtualized diff view (69c)" (still stale — see below), is not the "real measurement... not a synthetic microbenchmark" the issue asks for (the generator below is exactly that synthetic microbenchmark, by design — see "What this does NOT cover"), and was not landed after 69e as the issue recommends (69e hasn't landed; there is no such issue/PR in this repo as of this writing). Do not treat this section, its heading, or its commit (`ad7fba9`) as closing #211 — the issue tracks the remaining three items and should stay open until a future change wires `CumulativeHeights` into the render path and this budget is redone against that real shape.

Stale premise in #211's own text, corrected here rather than silently worked around. The issue asks to measure "the virtualized diff view (69c)." As of this writing that shape does not exist: the diff view renders `CommitDiff.patch` as one flat `<pre>` of every line (`crates/git-vista/src/detail.rs::accessible_patch_view`, mirrored full-screen in `viewer.rs`) — no virtualization is wired into it. #69c's `CumulativeHeights`/`visible_range` (`git_vista_core::virtualize`) is a real, already-tested primitive (`crates/git-vista-core/src/virtualize.rs`, 9 tests of its own from PR #179), but it has **zero consumers** anywhere in the tree — verified by grep, not assumed. There is nothing "virtualized diff view" shaped to benchmark yet; a budget claiming to cover one would be fiction.

What was measured instead, and why it's the honest substitute. `hunk_nav` (`crates/git-vista/src/features/diff/core.rs`) is real, host-tested (`features/diff/core.rs` is not wasm-gated — only `staging_view.rs` inside that module is), and runs on **every** diff render today regardless of virtualization: a full walk of the raw patch text that both `accessible_patch_view` (panel + full-screen viewer) and `staging_view.rs`'s hunk-selection labels call directly. The function's own doc comment already names the failure mode this budget pins down: "a 5 MB refactor diff can carry tens of thousands of hunks, and a rescan-per-hunk would stall the iPad's main thread before first paint." That shape doesn't exist today (both of `hunk_nav`'s passes are already O(n)) — this budget exists so it's caught immediately if a future edit reintroduces it.

What this does NOT cover — stated explicitly, not left implicit:

- The actual DOM/`<pre>` construction in `detail.rs`/`viewer.rs`. Both are `#[cfg(target_arch = "wasm32")]`-gated; `cargo test --workspace` never compiles them and this repo has no wasm test harness. Unmeasured, and no test here claims otherwise.
- Whether virtualization is "engaged." It isn't wired into the diff view at all, so there is nothing to prove engaged or broken — see above. A future task that wires `CumulativeHeights` into the render path should add its own budget alongside that wiring, not retrofit a claim onto this one.

- Real-world patch shapes (renames, binary markers, combined merge headers, uneven hunk sizes). The generator produces one synthetic file with uniformly-sized hunks — the cheapest-per-byte shape, deliberately mirroring 68e's uniform-untracked-file generator — so this is closer to a best case than a worst case.

Reproduce with:

```
cargo test -p git-vista -- --ignored --nocapture hunk_nav_ladder
```

(`#[ignore]`d — generates up to a ~7 MB synthetic patch, no place in every `cargo test`/CI run. See its doc comment in `features/diff/core.rs`.)

The ladder — hunk count vs. wall-clock time

Synthetic uniform hunks (`generate_patch`, 3 add/remove line pairs each), debug build, one run each, this host:

hunks	elapsed	patch bytes
100	0.482 ms	14,004
1,000	4.083 ms	141,804
2,000	8.294 ms	285,804
10,000	41.164 ms	1,437,804
20,000	85.035 ms	2,897,804
50,000	207.600 ms	7,277,804

Roughly linear (2,000 → 20,000 hunks, a 10x increase, cost 10.25x more time) — no evidence of a superlinear cost center at these sizes. The 2,000-hunk row sits just past `DIFF_PATCH_CAP` (200,000 bytes, the panel's patch cap in `handlers/read.rs`) — a realistic "hit the panel's cap" size, not an arbitrary round number. 50,000 hunks (~7 MB) sits past `DIFF_PATCH_CAP_FULL` (5,000,000 bytes, the full-screen viewer's cap) — the server would already have truncated a real patch this large before `hunk_nav` ever saw it; it's included to see the shape holds past both caps, not because a real request reaches it.

Stated budget

`hunk_nav` over a patch with up to 20,000 hunks (comparable to `DIFF_PATCH_CAP_FULL`) must complete in well under 1 second on hardware comparable to this host. The measured 85.0 ms at 20,000 hunks leaves roughly 11x headroom before that budget.

Regression tests, both in `features/diff/core.rs` (not `#[ignore]`d — run in every `cargo test`):

- `hunk_nav_budget_holds_at_2k_hunks` asserts 2,000 hunks (the panel-cap-realistic size above) complete inside **500 ms** — roughly 60x the measured 8.3 ms at that size.
- `hunk_nav_scales_roughly_linearly_not_quadratically` asserts the 20,000-hunk run costs less than **25x** the 2,000-hunk run's time. This is the test that actually catches the regression named above: an accidental reintroduction of a rescan-per-hunk shape

would show up as roughly a further 10x slowdown on top of the expected 10x (i.e. close to 100x for this 10x size increase) — the wall-clock budget alone would not reliably catch that until it got much worse, since 500 ms has a lot of headroom. Both tests also assert `hunk_nav` found exactly as many hunks as the generator produced, so a fast-but-wrong answer (an early return, or a desynced countdown eating the rest of the patch) fails the test instead of passing it by accident.